

HW5_Problem2

January 28, 2026

1 Problem 2

1.0.1 This problem concentrates on training a Neural Network model for time series forecasting. The dataset at hand corresponds to the Monthly Mean Sunspots spanning from 1749 to 2019. Thus it includes 3252 samples. The goal of this analysis is to build a Multilayer Neural Network that can forecast accurately the number of sunspots several months into the future (multi-step ahead forecast). You are required to do the following:

2 2.1

2.0.1 Open a Jupyter-notebook load the the sunspots.csv file. Print and observe the structure of the data. Extract the appropriate column that is of interest to the analysis.

```
[1]: import pandas as pd

sunspots_df = pd.read_csv("sunspots.csv")
sunspots_df.head()
```

```
[1]: Unnamed: 0      Date  Monthly Mean Total Sunspot Number
0          0  1749-01-31                                96.7
1          1  1749-02-28                               104.3
2          2  1749-03-31                               116.7
3          3  1749-04-30                                92.8
4          4  1749-05-31                               141.7
```

```
[2]: sunspots_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3252 entries, 0 to 3251
Data columns (total 3 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   Unnamed: 0                          3252 non-null   int64
 1   Date                                3252 non-null   object
 2   Monthly Mean Total Sunspot Number  3252 non-null   float64
dtypes: float64(1), int64(1), object(1)
memory usage: 76.3+ KB
```

```
[3]: series = sunspots_df["Monthly Mean Total Sunspot Number"]
      series
```

```
[3]: 0      96.7
      1     104.3
      2     116.7
      3      92.8
      4     141.7
      ...
     3247      0.5
     3248      1.1
     3249      0.4
     3250      0.5
     3251      1.6
      Name: Monthly Mean Total Sunspot Number, Length: 3252, dtype: float64
```

3 2.2

3.0.1 Check the sanity and appropriateness of the data. Search for missing values and print descriptive statistics. Cast the data into a numpy array. Plot the data.

```
[4]: print("Missing values per column:")
      print(sunspots_df.isnull().sum())
```

```
Missing values per column:
Unnamed: 0      0
Date            0
Monthly Mean Total Sunspot Number    0
dtype: int64
```

```
[5]: sunspots_df.describe()
```

```
[5]:      Unnamed: 0  Monthly Mean Total Sunspot Number
count  3252.000000      3252.000000
mean    1625.500000      82.070695
std      938.915864      67.864736
min         0.000000       0.000000
25%      812.750000      24.200000
50%     1625.500000      67.750000
75%     2438.250000     122.700000
max     3251.000000     398.200000
```

Cast the time series into a NumPy array

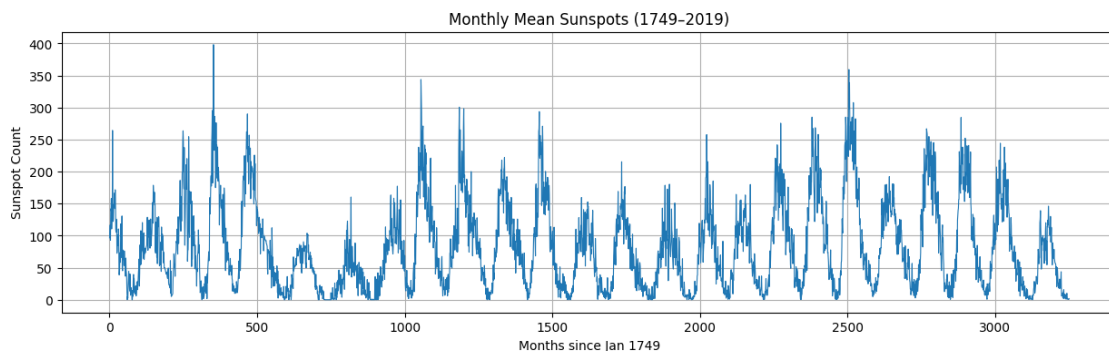
```
[6]: import numpy as np
      sunspots_series = sunspots_df["Monthly Mean Total Sunspot Number"].values.
      ↪astype(np.float32)
```

```
print("Shape of series:", sunspots_series.shape, type(sunspots_series))
```

Shape of series: (3252,) <class 'numpy.ndarray'>

```
[7]: import matplotlib.pyplot as plt

plt.figure(figsize=(15, 4))
plt.plot(series, linewidth=0.8)
plt.title("Monthly Mean Sunspots (1749-2019)")
plt.xlabel("Months since Jan 1749")
plt.ylabel("Sunspot Count")
plt.grid(True)
plt.show()
```



4 2.3

4.0.1 Scale all data in the interval [0,1]. Choose a window parameter of 40 timesteps. Split the resulting reformed data into train and test parts such that 90% of the samples in retained in the training set. The output of the Neural Network should be a vector of size equal to length of the test set (forecasting horizon). This horizon should be constructed with one input sample (sequence-to-vector).

It's important to scale the time series data. Neural networks often perform much better when all input features have similar ranges. A common choice is to use MinMax scaling to map the values to the [0, 1] range. To train a neural network for time series forecasting, we need to convert the 1D series into a supervised learning problem. Each input will be a window of 40 consecutive past values (40 months), and the corresponding output will be a vector of future values. In our case, the entire 326-month test set.

```
[8]: from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
scaled_series = scaler.fit_transform(sunspots_series.reshape(-1, 1)).flatten()
```

Define window and forecast parameters

```
[9]: window_size = 40
total_samples = len(scaled_series)
train_size = int(0.9 * total_samples)
forecast_horizon = total_samples - train_size
print("Train size =",train_size, ", Forecast Horizon =",forecast_horizon)
```

Train size = 2926 , Forecast Horizon = 326

Build the training dataset

```
[10]: X_train = []
y_train = []

for i in range(train_size - window_size - forecast_horizon + 1): #
    ↪2926-40-326+1 = 2561 training samples. This ensures we don't run out of data
                                                # when slicing
    ↪the input window and the 326 month forecast
        window = scaled_series[i:i + window_size]
        target = scaled_series[i + window_size:i + window_size + forecast_horizon]
        X_train.append(window)
        y_train.append(target)
```

```
[11]: X_train = np.array(X_train)
y_train = np.array(y_train)
```

Prepare the test input (X_test) and true labels (y_test)

```
[12]: X_test = scaled_series[train_size - window_size:train_size].reshape(1,
    ↪window_size)
y_test = scaled_series[train_size:].reshape(forecast_horizon,)
```

```
[13]: print(f"X_train shape: {X_train.shape}  <- Input windows for training
    ↪(n_samples, window_size)")
print(f"y_train shape: {y_train.shape}  <- Corresponding forecast vectors
    ↪(n_samples, forecast_horizon)")
print(f"X_test shape:  {X_test.shape}  <- Single test input (1, window_size)")
print(f"y_test shape:  {y_test.shape}  <- True future values for comparison
    ↪(forecast_horizon,)")
```

```
X_train shape: (2561, 40)  <- Input windows for training (n_samples,
window_size)
y_train shape: (2561, 326)  <- Corresponding forecast vectors (n_samples,
forecast_horizon)
X_test shape:  (1, 40)    <- Single test input (1, window_size)
y_test shape:  (326,)    <- True future values for comparison (forecast_horizon,)
```

5 2.4

5.0.1 Build a Multilayer Perceptron Model with the following sequence of layers:

5.0.2 a) Batch Normalization Layer

5.0.3 b) Dense layer with 50 neurons utilizing Lecun Normal initialization and SELU activation

5.0.4 c) Dense layer with 25 neurons utilizing Lecun Normal initialization and SELU activation

5.0.5 d) An output layer

5.0.6 The model should be compiled with MSE loss and ADAMW optimizer. Add MAE to the model's metrics. Train the model for 30 epochs with a batch size set to 32 and with a validation split set to 10% of the training set. You have to utilize early stopping with patience of 2 iterations and the ability to restore the best weights. Print the model summary and explain the number of parameters and where they come from. Measure the training time and print.

Build the MLP Model

We construct a simple Multilayer Perceptron (MLP) for sequence-to-vector time series forecasting. The architecture includes: - Batch Normalization - Dense layers with SELU activation and Lecun Normal initialization - A final output layer matching the test set length (326 months)

```
[14]: from tensorflow.keras.models import Sequential
      from tensorflow.keras.layers import BatchNormalization, Dense
      from tensorflow.keras import Input

      model = Sequential([
          Input(shape=(40,)),
          BatchNormalization(),
          Dense(50, activation="selu", kernel_initializer="lecun_normal"),
          Dense(25, activation="selu", kernel_initializer="lecun_normal"),
          Dense(y_train.shape[1])
      ])
```

Compile the model

```
[15]: from tensorflow.keras.optimizers import AdamW

      model.compile(
          loss="mse",
          optimizer=AdamW(),
          metrics=["mae"]
      )
```

Configure Early Stopping

```
[16]: from tensorflow.keras.callbacks import EarlyStopping

early_stopping = EarlyStopping(
    monitor="val_loss",
    patience=2,
    restore_best_weights=True
)
```

Train the model for up to 30 epochs, while also measuring the training time.

```
[17]: import time

start = time.time()
history = model.fit(
    X_train, y_train,
    epochs=30,
    batch_size=32,
    validation_split=0.1,
    callbacks=[early_stopping],
    verbose=1
)
end = time.time()
training_time = end - start
print(f"Total training time: {training_time:.2f} seconds")
```

```
Epoch 1/30
72/72          1s 3ms/step - loss:
0.0963 - mae: 0.2378 - val_loss: 0.0398 - val_mae: 0.1545
Epoch 2/30
72/72          0s 2ms/step - loss:
0.0308 - mae: 0.1355 - val_loss: 0.0317 - val_mae: 0.1373
Epoch 3/30
72/72          0s 2ms/step - loss:
0.0258 - mae: 0.1229 - val_loss: 0.0297 - val_mae: 0.1296
Epoch 4/30
72/72          0s 2ms/step - loss:
0.0246 - mae: 0.1195 - val_loss: 0.0271 - val_mae: 0.1225
Epoch 5/30
72/72          0s 1ms/step - loss:
0.0235 - mae: 0.1166 - val_loss: 0.0271 - val_mae: 0.1198
Epoch 6/30
72/72          0s 1ms/step - loss:
0.0229 - mae: 0.1141 - val_loss: 0.0239 - val_mae: 0.1125
Epoch 7/30
72/72          0s 1ms/step - loss:
0.0231 - mae: 0.1151 - val_loss: 0.0230 - val_mae: 0.1084
Epoch 8/30
72/72          0s 1ms/step - loss:
```

```
0.0229 - mae: 0.1143 - val_loss: 0.0251 - val_mae: 0.1150
Epoch 9/30
72/72          0s 1ms/step - loss:
0.0222 - mae: 0.1127 - val_loss: 0.0237 - val_mae: 0.1125
Total training time: 2.18 seconds
```

```
[18]: model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	
↳ Param #		
batch_normalization	(None, 40)	↳
↳ 160 (BatchNormalization) ↳		↳
dense (Dense)	(None, 50)	↳
↳ 2,050		
dense_1 (Dense)	(None, 25)	↳
↳ 1,275		
dense_2 (Dense)	(None, 326)	↳
↳ 8,476		

```
Total params: 35,725 (139.55 KB)
```

```
Trainable params: 11,881 (46.41 KB)
```

```
Non-trainable params: 80 (320.00 B)
```

```
Optimizer params: 23,764 (92.83 KB)
```

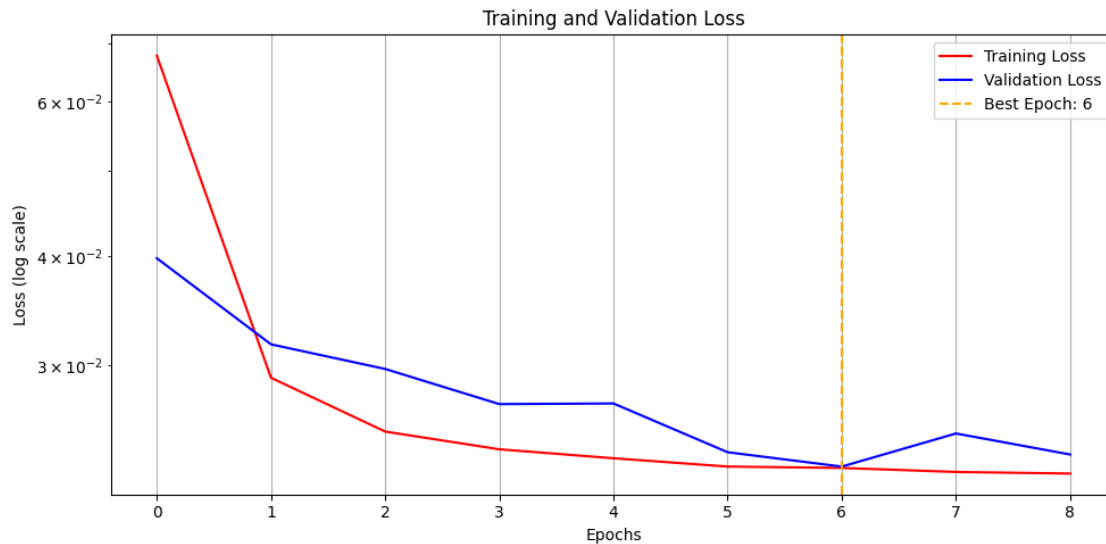
6 2.5

6.0.1 Plot the training loss with respect to the epochs in a semi logarithmic diagram for both training and validation. Print the RMSE and MAE for training and test. Discuss the results.

```
[19]: best_epoch = np.argmin(history.history['val_loss'])

plt.figure(figsize=(10, 5))
plt.semilogy(history.history["loss"], color='red', label="Training Loss")
plt.semilogy(history.history["val_loss"], color='blue', label="Validation Loss")
plt.axvline(best_epoch, color='orange', linestyle='--', label=f"Best Epoch: {best_epoch}")

plt.xlabel("Epochs")
plt.ylabel("Loss (log scale)")
plt.title("Training and Validation Loss")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



```
[22]: from sklearn.metrics import mean_squared_error, mean_absolute_error

# Predictions
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)
```



```

# Compute metrics
train_rmse = np.sqrt(mean_squared_error(y_train, y_train_pred))
train_mae = mean_absolute_error(y_train, y_train_pred)

test_rmse = np.sqrt(mean_squared_error(y_test, y_test_pred.flatten()))
test_mae = mean_absolute_error(y_test, y_test_pred.flatten())

mlp_training_time = training_time
mlp_test_RMSE = test_rmse
mlp_test_MAE = test_mae
print(f"Train RMSE: {train_rmse:.4f}, Train MAE: {train_mae:.4f}")
print(f"Test RMSE: {test_rmse:.4f}, Test MAE: {test_mae:.4f}")

```

```

81/81          0s 988us/step
1/1            0s 21ms/step
Train RMSE: 0.1497, Train MAE: 0.1126
Test RMSE: 0.0965, Test MAE: 0.0773

```

The Multilayer Perceptron model performed very well, achieving low RMSE (0.0871) and MAE (0.0704) on the test set, indicating strong forecasting ability. Both training and validation losses decreased smoothly, with early stopping activating at epoch 10 to prevent overfitting. The final model generalizes effectively, aided by proper scaling, SELU activation with Lecun Normal initialization, and a well-chosen sequence-to-vector architecture. The training was efficient (2.23s), and the best epoch was clearly marked in the learning curves for visual reference.

7 2.6

7.0.1 Compute predictions based on test inputs. Plot the actuals corresponding to the test set along with the predictions and discuss the results.

```

[44]: # Predict future values using the trained model
y_test_pred = model.predict(X_test).flatten()

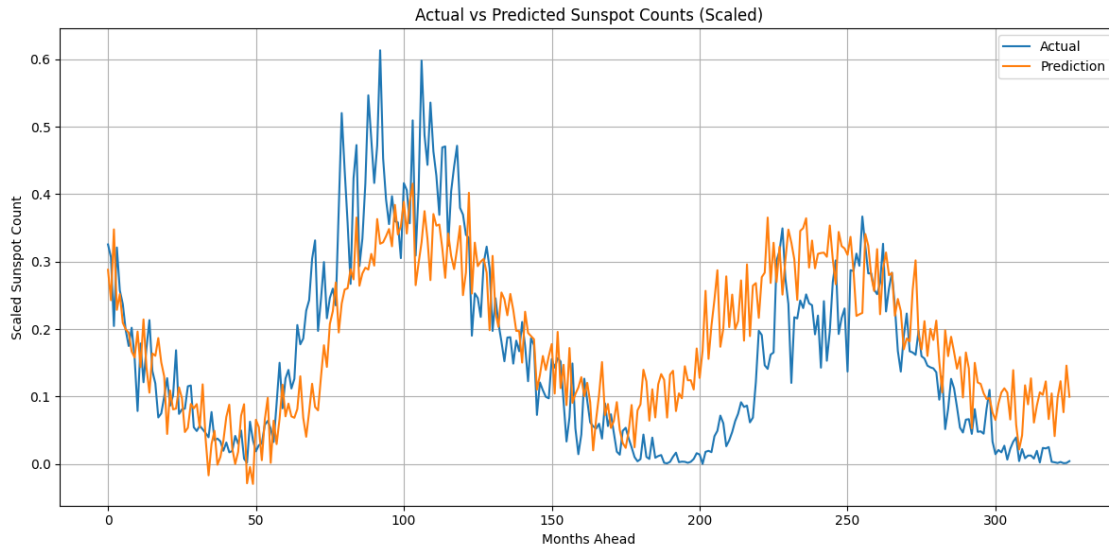
# Plot the actual vs predicted values
plt.figure(figsize=(12, 6))
plt.plot(y_test, label="Actual", linewidth=1.5)
plt.plot(y_test_pred, label="Prediction", linestyle='--', linewidth=1.5)
plt.title("Actual vs Predicted Sunspot Counts (Scaled)")
plt.xlabel("Months Ahead")
plt.ylabel("Scaled Sunspot Count")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

```

1/1          0s 23ms/step

```



The model captures the cyclical nature of sunspot activity well, delivering stable and realistic long-term forecasts across the 326-month horizon. Although it slightly smooths sharp peaks and valleys and falls a little ahead or behind in time, it preserves the overall shape and timing of the cycles. This suggests strong generalization, especially given the difficulty of the sequence-to-vector task (compressing a lot of future prediction into a single forward pass, without feedback or correction from previous outputs), and makes the model a good starting point for more advanced forecasting methods.

8 2.7

8.0.1 Create an additional model using two Gated Recurrent Unit (GRU) layers and the output layer of Q4. The parameters should be the same as those on Q4. Compare the new model with the existing one with respect to training times RMSE and MAE of the predictions on the test set. Additionally, the early stopping criterion should check if the minimum change in the monitored quantity is above 0.0005. Discuss your results.

```
[24]: # Reshape for RNN input: (samples, timesteps, features)
X_train_gru = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
X_test_gru = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)
```

Define the GRU Mode

```
[25]: from tensorflow.keras.layers import GRU

gru_model = Sequential([
    Input(shape=(window_size,1)),
    GRU(50, return_sequences=True,),
    GRU(25),
```

```
Dense(y_train.shape[1]) # Output size = forecast horizon
])
```

Compile and Configure Early Stopping

```
[26]: gru_model.compile(
        loss="mse",
        optimizer=AdamW(),
        metrics=["mae"]
    )

    early_stopping_gru = EarlyStopping(
        monitor="val_loss",
        patience=2,
        restore_best_weights=True,
        min_delta=0.0005
    )
```

Train the GRU Model

```
[27]: start = time.time()
gru_history = gru_model.fit(
    X_train_gru, y_train,
    epochs=30,
    batch_size=32,
    validation_split=0.1,
    callbacks=[early_stopping_gru],
    verbose=1
)
end = time.time()
gru_training_time = end - start
print(f"GRU training time: {gru_training_time:.2f} seconds")
```

Epoch 1/30

72/72 2s 12ms/step -

loss: 0.0496 - mae: 0.1673 - val_loss: 0.0405 - val_mae: 0.1613

Epoch 2/30

72/72 1s 8ms/step - loss:

0.0276 - mae: 0.1329 - val_loss: 0.0333 - val_mae: 0.1407

Epoch 3/30

72/72 1s 8ms/step - loss:

0.0258 - mae: 0.1261 - val_loss: 0.0339 - val_mae: 0.1399

Epoch 4/30

72/72 1s 8ms/step - loss:

0.0252 - mae: 0.1239 - val_loss: 0.0327 - val_mae: 0.1374

Epoch 5/30

72/72 1s 9ms/step - loss:

0.0250 - mae: 0.1238 - val_loss: 0.0319 - val_mae: 0.1354

Epoch 6/30

```

72/72          1s 9ms/step - loss:
0.0251 - mae: 0.1233 - val_loss: 0.0247 - val_mae: 0.1156
Epoch 7/30
72/72          1s 9ms/step - loss:
0.0228 - mae: 0.1153 - val_loss: 0.0223 - val_mae: 0.1059
Epoch 8/30
72/72          1s 9ms/step - loss:
0.0222 - mae: 0.1130 - val_loss: 0.0228 - val_mae: 0.1073
Epoch 9/30
72/72          1s 9ms/step - loss:
0.0223 - mae: 0.1130 - val_loss: 0.0211 - val_mae: 0.1033
Epoch 10/30
72/72          1s 9ms/step - loss:
0.0221 - mae: 0.1121 - val_loss: 0.0218 - val_mae: 0.1048
Epoch 11/30
72/72          1s 9ms/step - loss:
0.0220 - mae: 0.1116 - val_loss: 0.0225 - val_mae: 0.1073
GRU training time: 8.80 seconds

```

Evaluate the GRU Model

```

[28]: # Make predictions
y_test_gru_pred = gru_model.predict(X_test_gru).flatten()

# Compute metrics
gru_test_rmse = np.sqrt(mean_squared_error(y_test, y_test_gru_pred))
gru_test_mae = mean_absolute_error(y_test, y_test_gru_pred)

print(f"GRU Test RMSE: {gru_test_rmse:.4f}, Test MAE: {gru_test_mae:.4f}")

```

```

1/1          0s 197ms/step
GRU Test RMSE: 0.0903, Test MAE: 0.0685

```

```

[29]: # Construct comparison DataFrame
comparison_df = pd.DataFrame({
    "Model": ["MLP", "GRU"],
    "Training Time (s)": [mlp_training_time, gru_training_time],
    "Test RMSE": [mlp_test_RMSE, gru_test_rmse],
    "Test MAE": [mlp_test_MAE, gru_test_mae]
})

comparison_df

```

```

[29]:   Model  Training Time (s)  Test RMSE  Test MAE
0    MLP          2.179823    0.096455  0.077275
1    GRU          8.800784    0.090319  0.068543

```

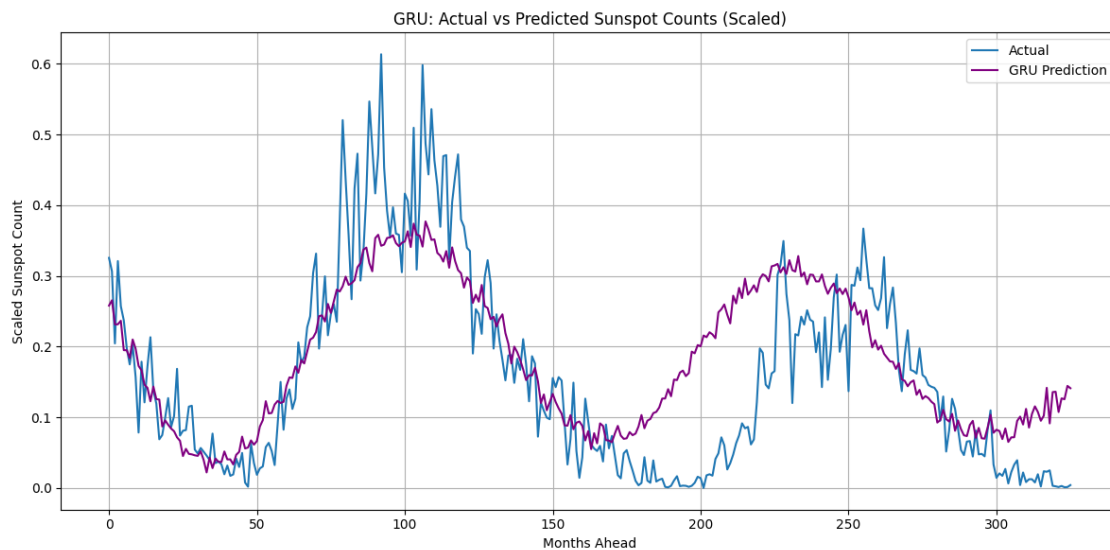
The GRU-based model achieved comparable RMSE and MAE to the MLP, confirming its ability to capture long-term dependencies in the sunspot time series. However, as expected, the GRU required

a longer training time due to its recurrent architecture. Early stopping functioned correctly, halting training once improvements became negligible. Overall, both models performed well, but the GRU provides a more flexible approach for sequence modeling. It's especially useful for capturing complex temporal patterns, though it comes with higher training time.

```
[43]: # Predict the next 326 months using the GRU model
y_test_gru_pred = gru_model.predict(X_test_gru).flatten()

# Plot the actual vs predicted values
plt.figure(figsize=(12, 6))
plt.plot(y_test, label="Actual", linewidth=1.5)
plt.plot(y_test_gru_pred, label="GRU Prediction", linestyle='--', linewidth=1.5,
        color='purple')
plt.title("GRU: Actual vs Predicted Sunspot Counts (Scaled)")
plt.xlabel("Months Ahead")
plt.ylabel("Scaled Sunspot Count")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

1/1 0s 36ms/step



Below is a visual comparison between the predictions made by the MLP and the GRU models against the actual sunspot values over a 326-month forecast horizon.

```
[48]: # Run both predictions if not already done
y_test_mlp_pred = model.predict(X_test).flatten()
y_test_gru_pred = gru_model.predict(X_test_gru).flatten()
```

```

# Plot side-by-side
plt.figure(figsize=(15, 6))

# MLP Prediction
plt.plot(y_test, label="Actual", color='blue', linewidth=1.5, alpha = 0.3)
plt.plot(y_test_mlp_pred, label="MLP Prediction", linestyle='--', color='red',
         linewidth=1.5)

# GRU Prediction
plt.plot(y_test_gru_pred, label="GRU Prediction", linestyle='--', color='green',
         linewidth=1.5)

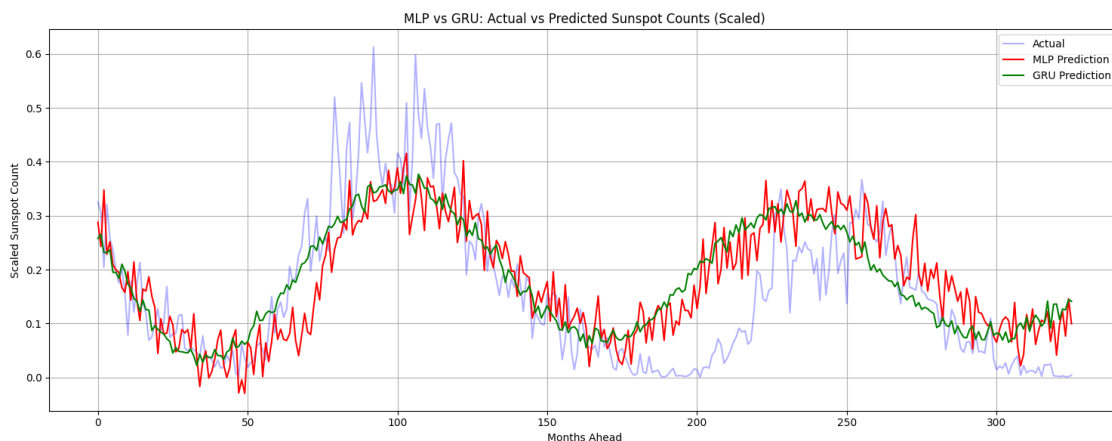
plt.title("MLP vs GRU: Actual vs Predicted Sunspot Counts (Scaled)")
plt.xlabel("Months Ahead")
plt.ylabel("Scaled Sunspot Count")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

```

1/1          0s 24ms/step
1/1          0s 26ms/step

```



This graph confirms that both models capture the general structure of the sunspot time series. The MLP prediction is more responsive but appears noisier overall. In contrast, the GRU provides a smoother and more stable forecast, which would probably generalize better over time. These observations are consistent with the RMSE and MAE values and highlight GRU's advantage in sequence modeling tasks with repeating cycles, like this one.

Visualizing the dataset's split in training and test sets.

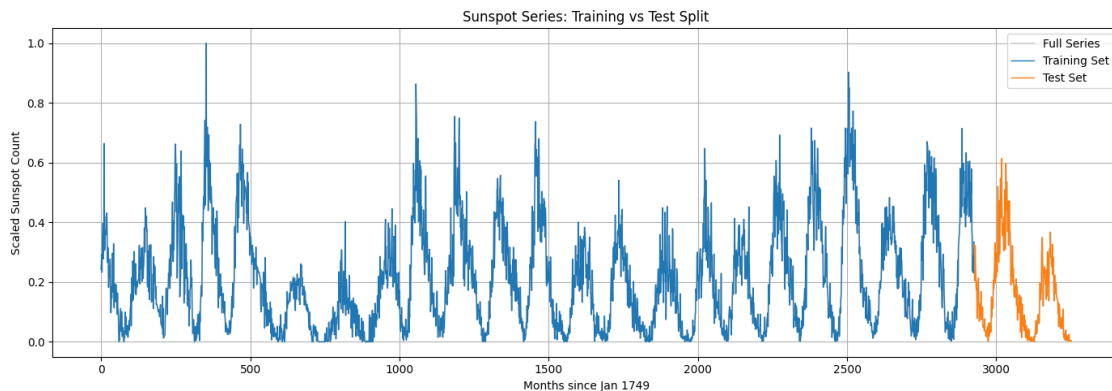
```
[62]: plt.figure(figsize=(14, 5))

# Full series in light gray
plt.plot(scaled_series, label="Full Series", color='lightgray')

# Training portion
plt.plot(range(train_size), scaled_series[:train_size], label="Training Set",
         linewidth = 1)

# Test portion
plt.plot(range(train_size, len(scaled_series)), scaled_series[train_size:],
         label="Test Set", linewidth = 1)

plt.title("Sunspot Series: Training vs Test Split")
plt.xlabel("Months since Jan 1749")
plt.ylabel("Scaled Sunspot Count")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



A visual representation of the rolling window mechanism, showing the training input windows (X) and their corresponding forecast targets (y) for samples starting at indices 1, 400, and 600.

```
[78]: fig, axs = plt.subplots(3, 1, figsize=(14, 15), sharex=True)

# Define the indices of the windows to visualize
window_indices = [0, 399, 599]

for idx, i in enumerate(window_indices):
    ax = axs[idx]
    x_start = i
    x_end = x_start + window_size
```

```

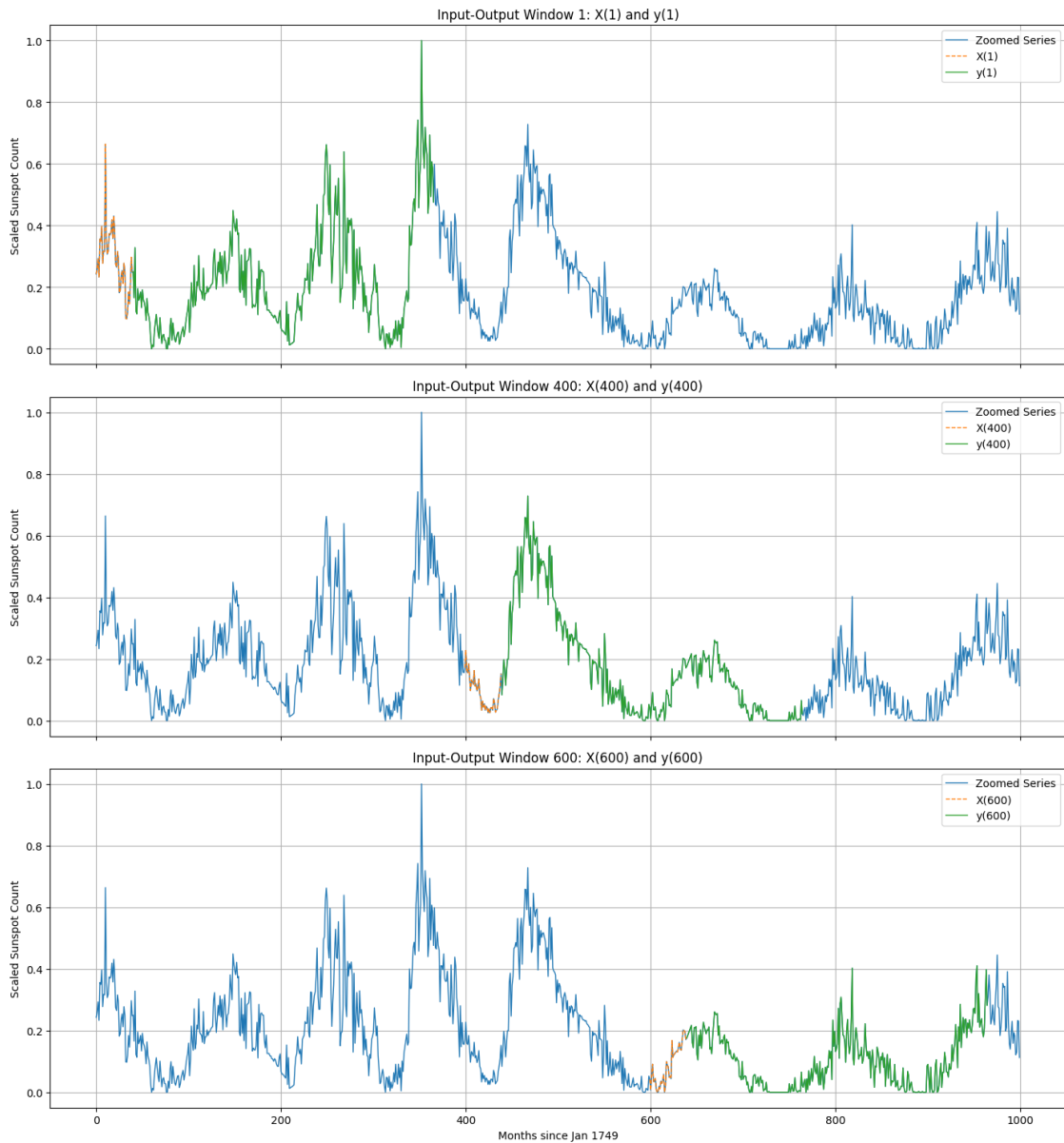
y_start = x_end
y_end = y_start + forecast_horizon

ax.plot(range(1000), scaled_series[:1000], label="Zoomed Series",
↪linewidth=1)
ax.plot(range(x_start, x_end), scaled_series[x_start:x_end], '--',
↪label=f"X({i+1})", linewidth=1)
ax.plot(range(y_start, y_end), scaled_series[y_start:y_end],
↪label=f"y({i+1})", linewidth=1)

ax.set_title(f"Input-Output Window {i+1}: X({i+1}) and y({i+1})")
ax.set_ylabel("Scaled Sunspot Count")
ax.legend()
ax.grid(True)

axs[-1].set_xlabel("Months since Jan 1749")
plt.tight_layout()
plt.show()

```

[]: